Estructura de un aplicación en Spring

Desarrollo Entorno Servidor

Spring

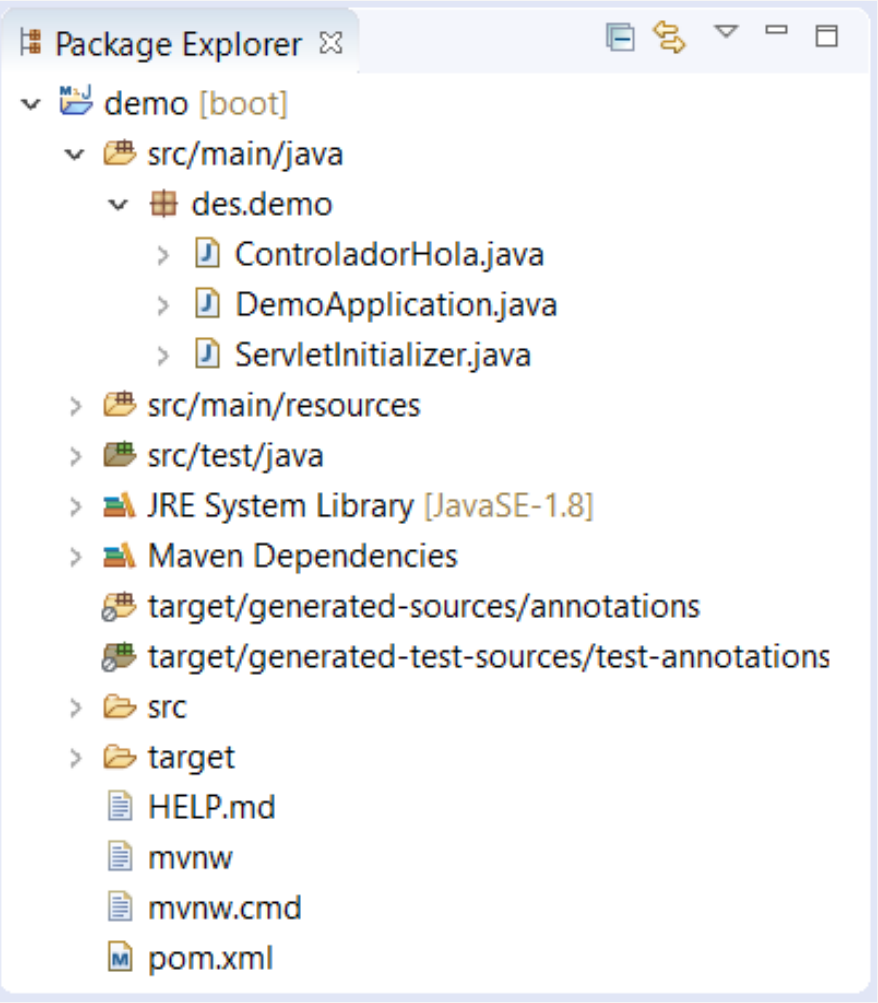
El objetivo de estas diapositivas es de explicar la estructura básica de un proyecto Spring, y así establecer una organización en la forma de trabajo.

Es importante entender que en un proyecto software que todo tiene su sitio o se le debe buscar uno, ya que a medida que el proyecto incrementa su tamaño y su complejidad aumenta proporcionalmente la dificultad de trabajar en el.

Es importante entender que los proyectos son realizados en equipo, por ello es importante entender que la responsabilidad de mantener la organización de proyecto es colectiva.

Estructura proyecto

Esta es la estructura de un proyecto en Eclipse. Sobre esto guiaremos este documento.



Pom.xml

POM (Proyect Object Model) es un fichero en formato XML que almacena información del proyecto y configuración, que se utiliza para para describir el proyecto de software a construir, sus dependencias de otros módulos y componentes externos, y el orden de construcción de los elementos.

Este fichero es empleado por Maven.

Apache Maven es una herramienta que estandariza la configuración de un proyecto en todo su ciclo de vida, como por ejemplo en todas las fases de compilación y empaquetado y la instalación de mecanismos de distribución de librerías, para que puedan ser utilizadas por otros desarrolladores y equipos de desarrollo.

Pom.xml

POM (Proyect Object Model) es un fichero en formato XML que almacena información del proyecto y configuración.

demo/pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.3.3.RELEASE</version>
        <relativePath /> <!-- lookup parent from repository -->
    </parent>
    <groupId>des</groupId>
    <artifactId>demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>war</packaging>
    <name>demo</name>
    <description>empezando con Spring Boot</description>

    <properties>
        <java.version>1.8</java.version>
    </properties>
```

Información del proyecto

Pom.xml

POM (Proyect Object Model) es un fichero en formato XML que almacena información del proyecto y configuración.

demo/pom.xml

```xml
23    <dependencies>
24        <dependency>
25            <groupId>org.springframework.boot</groupId>
26            <artifactId>spring-boot-starter-thymeleaf</artifactId>
27        </dependency>
28
29        <dependency>
30            <groupId>org.springframework.boot</groupId>
31            <artifactId>spring-boot-starter-web</artifactId>
32        </dependency>
33
34        <dependency>
35            <groupId>org.springframework.boot</groupId>
36            <artifactId>spring-boot-starter-test</artifactId>
37            <scope>test</scope>
38            <exclusions>
39                <exclusion>
40                    <groupId>org.junit.vintage</groupId>
41                    <artifactId>junit-vintage-engine</artifactId>
42                </exclusion>
43            </exclusions>
44        </dependency>
45    </dependencies>
```

Librerías de las que depende.

Package Explorer

Maven Dependencies

- spring-boot-starter-thymeleaf-2.3.3.RELEASE.jar
- spring-boot-starter-2.3.3.RELEASE.jar - C:\Users\r
- spring-boot-2.3.3.RELEASE.jar - C:\Users\marco\.
- spring-boot-autoconfigure-2.3.3.RELEASE.jar - C:
- spring-boot-starter-logging-2.3.3.RELEASE.jar - C
- logback-classic-1.2.3.jar - C:\Users\marco\.m2\re
- logback-core-1.2.3.jar - C:\Users\marco\.m2\repo
- log4j-to-slf4j-2.13.3.jar - C:\Users\marco\.m2\rep
- log4j-api-2.13.3.jar - C:\Users\marco\.m2\reposit
- jul-to-slf4j-1.7.30.jar - C:\Users\marco\.m2\repos
- jakarta.annotation-api-1.3.5.jar - C:\Users\marco\
- snakeyaml-1.26.jar - C:\Users\marco\.m2\reposit
- thymeleaf-spring5-3.0.11.RELEASE.jar - C:\Users\
- thymeleaf-3.0.11.RELEASE.jar - C:\Users\marco\.n
- attoparser-2.0.5.RELEASE.jar - C:\Users\marco\.m
- unbescape-1.1.6.RELEASE.jar - C:\Users\marco\.m

Que se descargan en el equipo.

Pom.xml

POM (Proyect Object Model) es un fichero en formato XML que almacena información del proyecto y configuración.

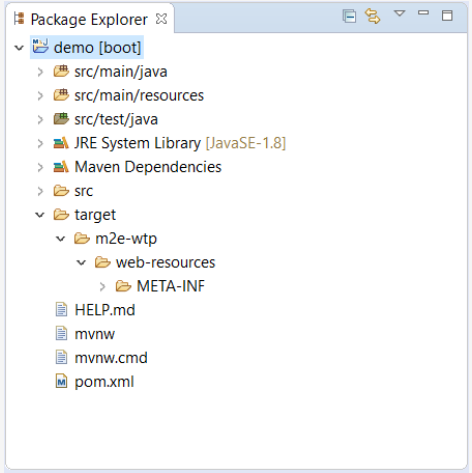
```
47    <build>
48        <plugins>
49            <plugin>
50                <groupId>org.springframework.boot</groupId>
51                <artifactId>spring-boot-maven-plugin</artifactId>
52            </plugin>
53        </plugins>
54    </build>
```

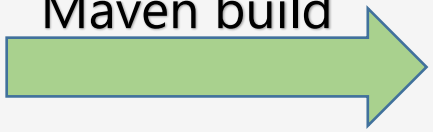
Que ocurre en la fase de construcción de Maven.

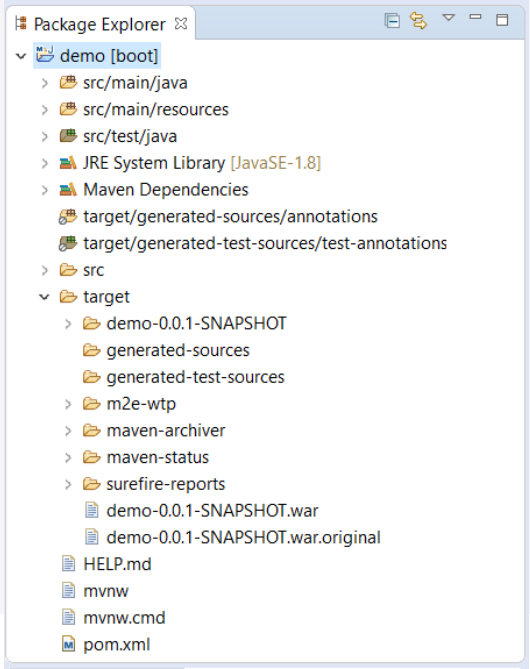
Target

Cada vez que hacemos las fase **Build de Maven** las librerías escritas en el pom.xml son descargadas y guardados en el directorio Target.

En ciertas ocasiones pueden generar error por las dependencias debido a actualizaciones sufridas por el proyecto es importante por ello borrar con las fase **clean** de **Maven** y volverlas a generar.

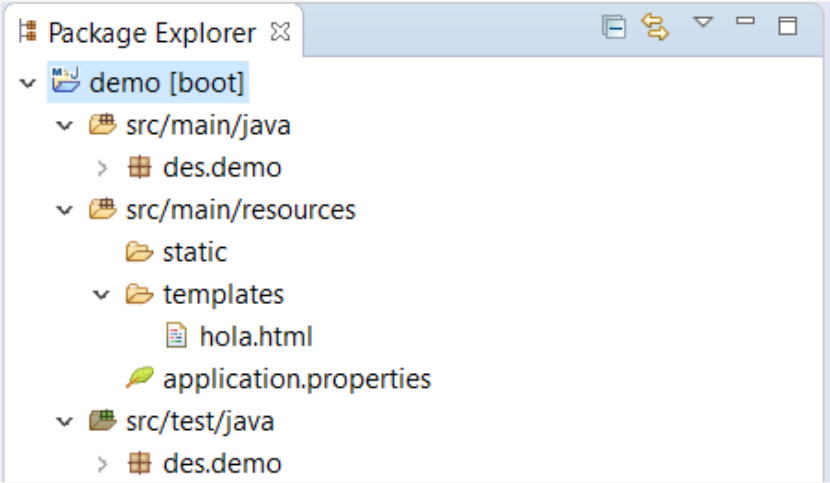






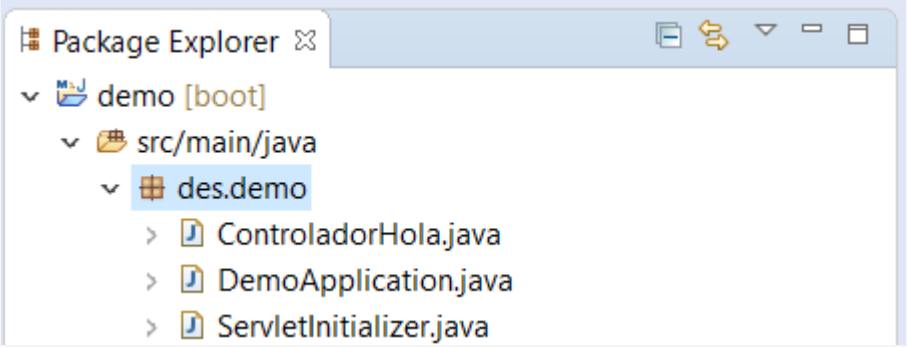
Directorios importantes

- **src/main/java** es el directorio donde reside el código java.
  - Importante la estructura de paquetes **des.demo** se corresponde con los atributos **groupId** y **artifactId** de nuestro **pom.xml**
- **src/main/resources** es donde residen los ficheros de configuración importantes
  - **static:** es donde se almacena el contenido estático es las hojas de estilo css, imágenes de cabecera.
  - **templates:** guarda los ficheros .html es decir las páginas de nuestra aplicación.
  - **application.properties:** es un fichero de configuración de Spring, este es leído antes de ejecutarse la aplicación.
- **src/test/java** directorio para las pruebas de nuestra aplicación.

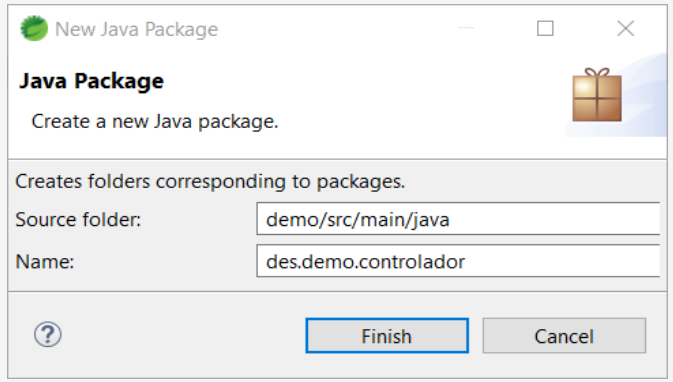


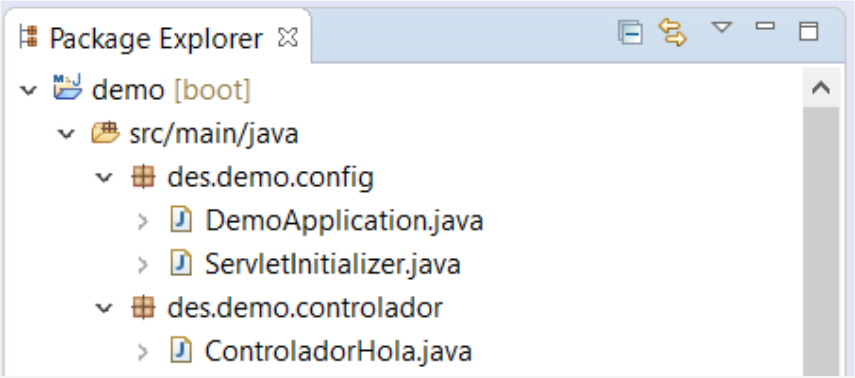
src/main/java

En este directorio encontraremos:



Debemos separar esto en paquetes ya que a medida que crezca la aplicación esta organización será insostenible.



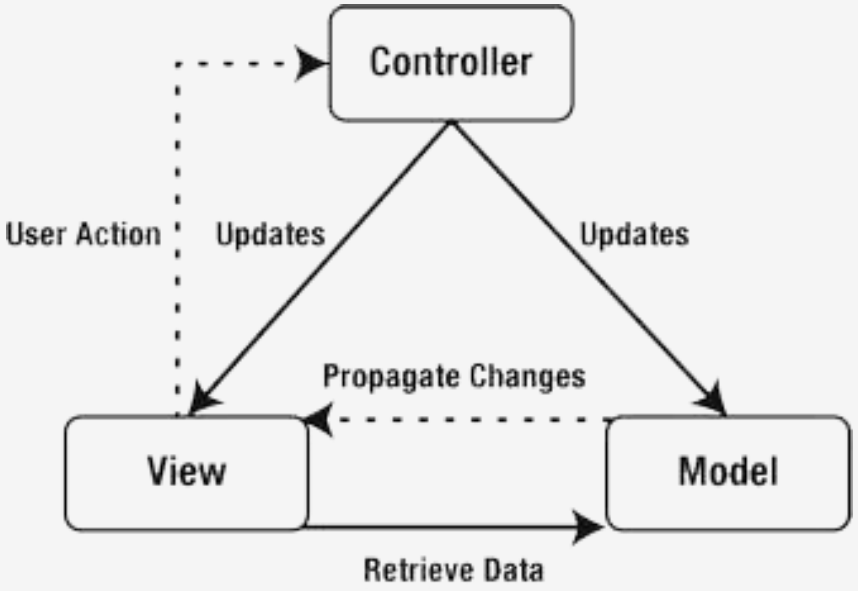


MVC Modelo Vista Controlador

Este patrón divide la capa de presentación en diferentes tipos de componentes.

Cada componente tiene sus propias responsabilidades:

- La vista utiliza el modelo para representarse a sí mismo.
- Según una acción del usuario, la vista activa el controlador, que a su vez actualiza el modelo.
- El modelo luego notifica la vista para (re) renderizarse.



MVC Modelo Vista Controlador

El patrón MVC tiene que ver con la separación de las responsabilidades. Como mencionamos anteriormente, cada componente tiene su propio rol. Seguir este enfoque mantiene todo bien separado, lo que facilita el mantenimiento y la prueba de nuestra aplicación.

- **Modelo:** El modelo son los datos que necesita la vista para poder representarlos. Puede ser un pedido realizado o una lista de libros solicitados por un usuario.

- **Vista:** La vista es la implementación real y utiliza el modelo para representarse en una aplicación web. Esto podría ser una página HTML, pero también podría ser una representación PDF o XML de una página.

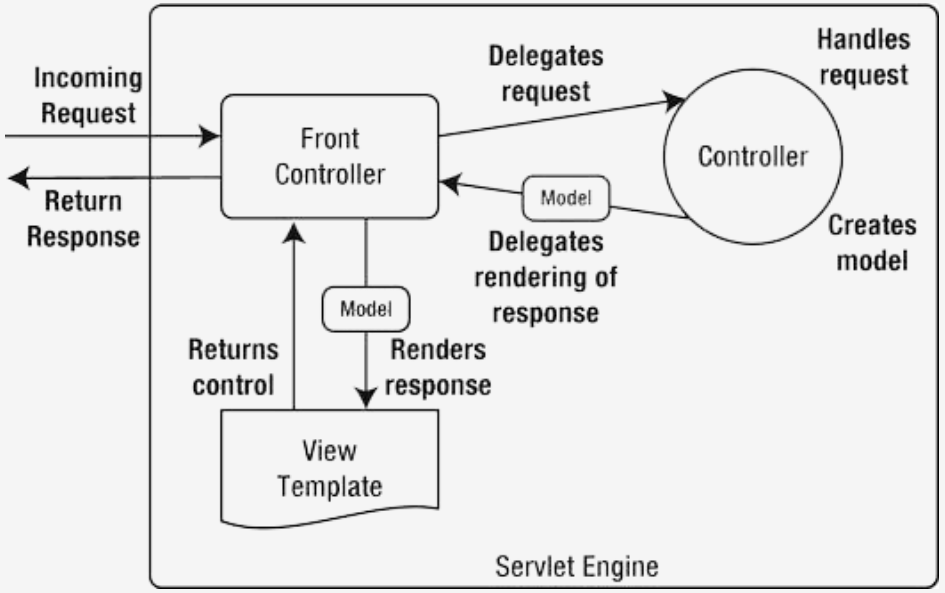
- **Controlador:** El controlador es el componente responsable de responder a la acción que realiza el usuario, como enviar un formulario o hacer clic en un enlace. El controlador actualiza el modelo y toma otras acciones necesarias, como invocar un método de servicio para realizar un pedido.

MVC Modelo Vista Controlador

La implementación clásica del patrón MVC implica que el usuario active una acción. Esto hace que el controlador actualice el modelo, lo que a su vez empuja los cambios a la vista. La vista luego se actualiza con los datos actualizados del modelo. Esta es la implementación ideal de un patrón MVC, y funciona muy bien en aplicaciones de escritorio basadas en Swing, por ejemplo. Sin embargo, este enfoque no es factible en un entorno web debido a la naturaleza del protocolo HTTP.

En una aplicación web, el usuario generalmente inicia una acción emitiendo una solicitud. Esto ocasiona que la aplicación se actualice y renderizar la vista, que se envía de vuelta al usuario. Esto significa que necesitamos un enfoque ligeramente diferente en un entorno web. En lugar de enviar los cambios a la vista, necesitamos extraer los cambios del servidor. Para ello emplearemos el controlador frontal.

MVC Modelo Vista Controlador
Controlador Frontal

El controlador frontal es el componente que maneja las solicitudes entrantes.

- Primero, delega la solicitud a un controlador adecuado.
- Cuando ese controlador haya terminado de procesar y actualizar el modelo, el controlador frontal determinará qué vista renderizar en función del resultado.